2

Getting to Know Node.js and MongoDB

In the previous chapter, we set up our IDE and a basic project for front-end development. In this chapter, we will first learn how to write and run scripts with Node.js. Then, we will move on to introducing Docker as a way to set up a database service. Once we have set up Docker and a container for our database, we are going to access it to learn more about MongoDB, the document database that we will use going forward. Finally, we will connect everything we have learned in this chapter by accessing MongoDB via Node.js scripts.

By the end of this chapter, you will have an understanding of the most important tools and concepts in backend development with JavaScript. This chapter gives us a good foundation to create a backend service for our first full-stack application in the upcoming chapters.

In this chapter, we are going to cover the following main topics:

- Writing and running scripts with Node.js
- Introducing Docker, a platform for containers
- Introducing MongoDB, a document database
- Accessing the MongoDB database via Node.js

Technical requirements

Before we start, please install the following (in addition to all technical requirements from ***Chapter 1***, *Preparing for Full-stack Development*), if you do not already have them installed:

- Docker v24.0.6
- Docker Desktop v4.25.2
- MongoDB Shell v2.1.0

The versions listed are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might

work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the mentioned versions.

You can find the code for this chapter on GitHub: **https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch2**.

The CiA video for this chapter can be found at: **https://youtu.be/q_LHsdJEaPo**.

IMPORTANT

If you cloned the full repository for the book, Husky may not find the `.git` *directory when running* `npm install`*. In that case, just run* `git init` *in the root of the corresponding chapter folder.*

Writing and running scripts with Node.js

For us to become full-stack developers, it is important to get familiar with backend technologies. As we are already familiar with JavaScript from writing frontend applications, we can use Node.js to develop backend services using JavaScript. In this section, we are going to create our first simple Node.js script to get familiar with the differences between backend scripts and frontend code.

Similarities and differences between JavaScript in the browser and in Node.js

Node.js is built on V8, the JavaScript engine used by Chromium-based browsers (Google Chrome, Brave, Opera, Vivaldi, and Microsoft Edge). As such, JavaScript code will run the same way in the browser and Node.js. However, there are some differences, specifically in the environment. The environment is built on top of the engine and allows us to render something on a website in the browser (using the `document` and `window` objects). In Node.js, there are certain modules provided to interface with the operating system, for tasks such as creating files and handling network requests. These modules allow us to create a backend service using Node.js.

Let's have a look at the Node.js architecture versus JavaScript in the browser:

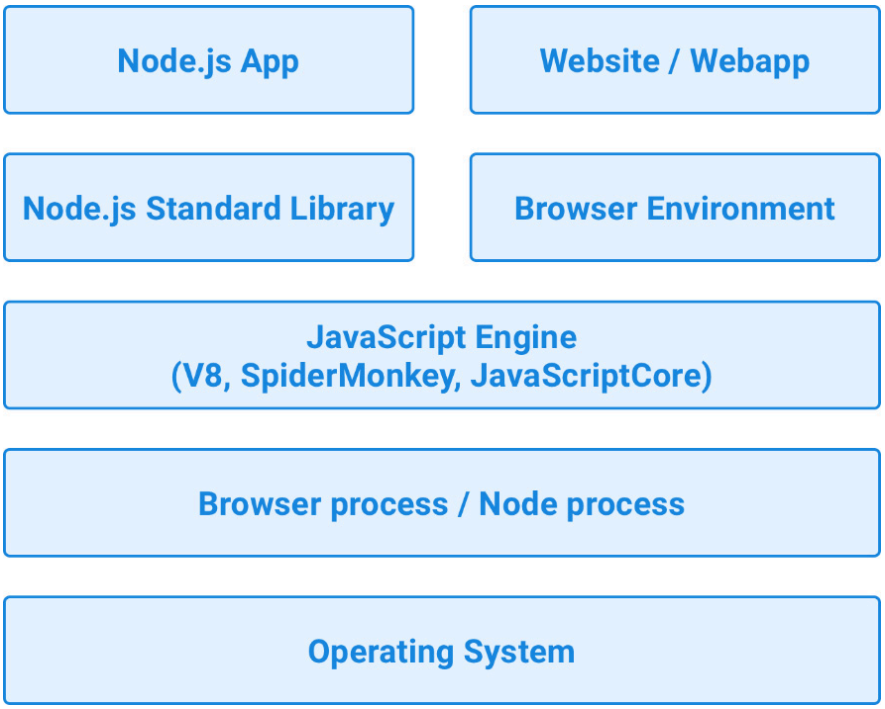

Figure 2.1 – The Node.js architecture versus JavaScript in the browser

As we can see from the visualization, both Node.js and browser JavaScript run on a JavaScript engine, which is always V8 in Node.js, and can be V8 for Chromium-based browsers, SpiderMonkey for Firefox, or JavaScriptCore for Safari.

Now that we know that we can run JavaScript code in Node.js, let's try it out!

Creating our first Node.js script

Before we can start writing backend services, we need to get familiar with the Node.js environment. So, let's start by writing a simple "hello world" example:

1. Copy the `ch1` folder from the previous chapter to a new `ch2` folder, as follows:

```
$ cp -R ch1 ch2
```

NOTE

On macOS, it is important to run the command with a capitalized `-R` *flag, not* `-r`*. The* `-r` *flag deals differently with symlinks and causes the* `node_modules/` *folder to break. The* `-r` *flag only exists for historic reasons and should not be used on macOS. Always prefer using the* `-R` *flag instead.*

2. Open the new **`ch2`** folder in VS Code.
3. Create a new **`backend`** folder in the **`ch2`** folder. This will contain our backend code.
4. In the **`backend`** folder, create a **`helloworld.js`** file and enter the following code:

```
console.log('hello node.js world!')
```

5. Open a Terminal in the **`ch2`** folder and run the following command to execute the Node.js script:

```
$ node backend/helloworld.js
```

You will see that the console output shows **`hello node.js world!`**. When writing Node.js code, we can make use of familiar functions from the frontend JavaScript world and run the same JavaScript code on the backend!

NOTE

While most frontend JavaScript code will run just fine in Node.js, not all code from the frontend will automatically work in a Node.js environment. There are certain objects, such as **`document`** *and* **`window`***, that are specific to a browser environment. This is important to keep in mind, especially when we introduce server-side rendering later.*

Now that we have a basic understanding of how Node.js works, let's get started handling files with Node.js.

Handling files in Node.js

Unlike in the browser environment, Node.js provides functions to handle files on our computer via the **`node:fs`** (filesystem) module. For example, we could make use of this functionality to read and write various files or even use files as a simple database.

Follow these steps to create your first Node.js script that handles files:

1. Create a new **`backend/files.js`** file.
2. Import the **`writeFileSync`** and **`readFileSync`** functions from the **`node:fs`** internal Node.js module. This module does not need to be installed via npm, as it is provided by the Node.js runtime.

```
import { writeFileSync, readFileSync } from 'node:fs'
```

3. Create a simple array containing users, with a name and email address:

```
const users = [{ name: 'Adam Ondra', email: 'adam.ondra@climb.ing' }]
```

4. Before we can save this array to a file, we first need to convert it to a string by using **`JSON.stringify`**:

```
const usersJson = JSON.stringify(users)
```

5. Now we can save our JSON string to a file by using the **`writeFileSync`** function. This function takes two arguments – first the filename, then the string to be written to the file:

```
writeFileSync('backend/users.json', usersJson)
```

6. After writing to the file, we can attempt reading it again using **`readFileSync`** and parsing the JSON string using **`JSON.parse`**:

```
const readUsersJson = readFileSync('backend/users.json')
const readUsers = JSON.parse(readUsersJson)
```

7. Finally, we log the parsed array:

```
console.log(readUsers)
```

8. Now we can run our script. You will see that the array gets logged and a **`users.json`** file was created in our **`backend/`** folder:

```
$ node backend/files.js
```

You may have noticed that we have been using **`writeFileSync`**, and not **`writeFile`**. The default behavior in Node.js is to run everything asynchronously, which means that if we used **`writeFile`**, the file may not have been created yet at the time when we called **`readFile`**, as asynchronous code is not executed in order.

This behavior might be annoying when writing simple scripts like we did, but is very useful when dealing with, for example, network requests, where we do not want to block other users from accessing our service while dealing with another request.

After learning about handling files with Node.js, let's learn more about how asynchronous code is executed in the browser and Node.js.

Concurrency with JavaScript in the browser and Node.js

An essential and special trait of JavaScript is that most API functions are asynchronous by default. This means that code does not simply run in the sequence in which it is defined. Specifically, JavaScript is event-driven. In the browser, this means that JavaScript code will run because of user interactions. For example, when a button is clicked, we define an `onClick` handler to execute some code.

On the server side, input/output operations, such as reading and writing files, and network requests, are handled asynchronously. This means that we can handle multiple network requests at once, without having to deal with threads or multiprocessing ourselves. Specifically, in Node.js, `libuv` is responsible for assigning threads for I/O operations while giving us, as a programmer, access to a single runtime thread to write our code in. However, this does not mean that each connection to our backend will create a new thread. Threads are created on the fly when advantageous. As a developer, we do not have to deal with multi-threading and can focus on developing with asynchronous code and callbacks.

If code is synchronous, it is executed directly by putting it on the **call stack**. If code is asynchronous, the operation is started, and the instance of that operation is stored in a queue, together with a callback function. The Node.js runtime will first execute all code left in the stack. Then, the **event loop** will come in and check whether there are any completed tasks in the queue. If that is the case, the callback function is executed by putting it on the stack. A callback function can then again either execute synchronous or asynchronous code. When we add an event listener – for example, an `onClick` listener in the browser – when the user clicks the related element, the callback will also be put in the task queue, which means it will be executed when nothing else is left on the stack. Similarly, in Node.js, we can add listeners for network events, and execute a callback when a request comes in.

In contrast to multi-threaded servers, a Node.js server accepts all requests in a single thread, which contains the event loop. Multi-threaded servers have the disadvantage that threads can block I/O completely and slow down the server. Node.js, however, delegates operations in a fine-grained way on the fly to threads. This results in less blocking of I/O operations by default. The downside with Node.js is that we have less control over how the multi-threading happens and thus need to be careful to avoid using synchronous functions whenever possible. Otherwise, we will block the main Node.js thread and slow down our server. For simplicity, we still use synchronous functions in this chapter. Going forward, in the next chap-

ters, we will avoid using those and rely solely on asynchronous functions (when possible) to get the best performance.

The following diagram visualizes the difference between multi-threaded servers and a Node.js server:

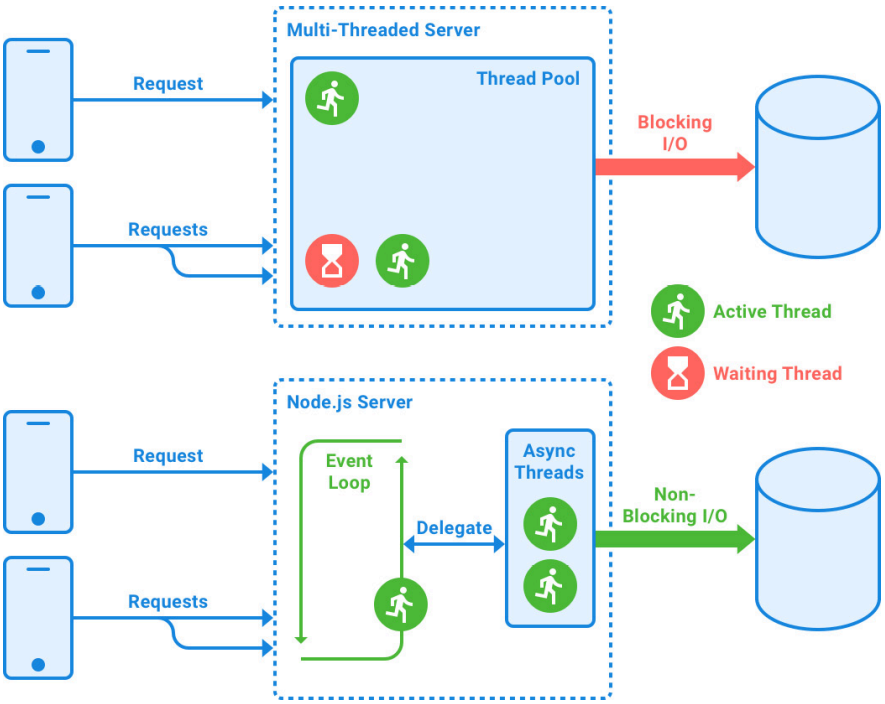

Figure 2.2 – The difference between multi-threaded servers and a Node.js server

We can see this asynchrony in action by using **`setTimeout`**, a function that you may be familiar with from frontend code. It waits a specified number of milliseconds and then executes the code specified in the callback function. For example, if we run the following code (with a Node.js script or in the browser, the result is the same for both):

```
console.log('first')
setTimeout(() => {
  console.log('second')
}, 1000)
console.log('third')
```

We can see that they get printed in the following order:

```
first
third
second
```

This makes sense, because we are delaying the "second" **`console.log`** by a second. However, the same output will happen if we execute the following code:

```
console.log('first')
setTimeout(() => {
  console.log('second')
}, 0)
console.log('third')
```

Now that we are waiting zero milliseconds before executing the code, you would think that "second" gets printed after "first." However, that is not the case. Instead, we get the same output as before:

```
first
third
second
```

The reason is that when we use **`setTimeout`**, the JavaScript engine calls either a web API (on the browser) or a native API (on Node.js). This API runs in native code in the engine, tracks the timeout internally, and puts the callback into the task queue, because the timer completes right away. While this is happening, the JavaScript engine continues processing the other code by pushing it onto the stack and executing it. When the stack is empty (there is no more code to execute), the event loop advances. It sees that there is something in the task queue, so it executes that code, resulting in "second" being printed last.

TIP

You can use the Loupe tool to visualize the inner workings of the Call Stack, web APIs, Event Loop, and Callback/Task Queue:
http://latentflip.com/loupe/

Now that we have learned how asynchronous code is handled in the browser and Node.js, let's create our first web server with Node.js!

Creating our first web server

Now that we have learned the basics of how Node.js works, we can use the **`node:http`** library to create a simple web server. For our first simple server, we are just going to return a **200 OK** response and some plain text on any request. Let's get started with the steps:

1. Create a new **`backend/simpleweb.js`** file, open it, and import the **`createServer`** function from the **`node:http`** module:

```
import { createServer } from 'node:http'
```

2. The **`createServer`** function is asynchronous, so it requires us to pass a callback function to it. This function will be executed when a request comes in from the server. It has two arguments, a request object (**`req`**) and a response object (**`res`**). Use the **`createServer`** function to define a new server:

```
const server = createServer((req, res) => {
```

3. For now, we will ignore the request object and only return a static response. First, we set the status code to **`200`**:

```
  res.statusCode = 200
```

4. Then, we set the **`Content-Type`** header to **`text/plain`**, such that the browser knows what kind of response data it is dealing with:

```
  res.setHeader('Content-Type', 'text/plain')
```

5. Lastly, we end the request by returning a **`Hello HTTP world!`** string in the response:

```
  res.end('Hello HTTP world!')
})
```

6. After defining the server, we need to make sure to listen on a certain host and port. These will define where the server will be available. For now, we use localhost on port **`3000`** to make sure our server is available via **`http://localhost:3000/`**:

```
const host = 'localhost'
const port = 3000
```

7. The **`server.listen`** function is also asynchronous and requires us to pass a callback function, which will execute as soon as the server is up and running. We can simply log something here for now:

```
server.listen(port, host, () => {
  console.log(`Server listening on http://${host}:${port}`)
})
```

8. Run the Node.js script as follows:

```
$ node backend/simpleweb.js
```

9. You will notice that we get our **Server listening on http://localhost:3000** log message, so we know the server was started successfully. This time, the Terminal does not return control to us; the script keeps running. We can now open **`http://localhost:3000`** in a browser:

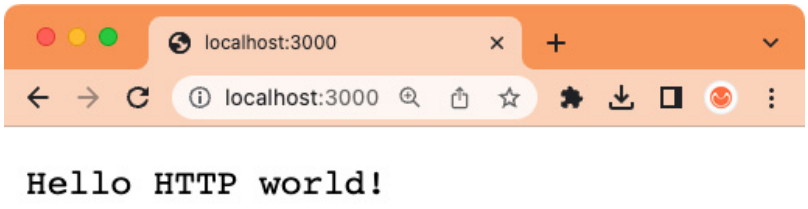

Figure 2.3 – A plaintext response from our first web server!

Now that we have set up a simple web server, we can extend it to serve a JSON file instead of simply returning plaintext.

Extending the web server to serve our JSON file

We can now try combining our knowledge of the **`node:fs`** module with the HTTP server to create a server that serves the previously created **`users.json`** file. Let's get started with the steps:

1. Copy the **`backend/simpleweb.js`** file to a new **`backend/webfiles.js`** file.
2. At the beginning of the file, add an import of **`readFileSync`**:

```
import { readFileSync } from 'node:fs'
```

3. Change the **`Content-Type`** header to **`application/json`**:

```
  res.setHeader('Content-Type', 'application/json')
```

4. Replace the string in **`res.end()`** with the JSON string from our file. In this case, we do not need to parse the JSON, as **`res.end()`** expects a string anyway:

```
  res.end(readFileSync('backend/users.json'))
```

5. If it is still running, stop the previous server script via *Ctrl* + *C*. We need to do this because we cannot listen on the same port twice.
6. Run the server and refresh the page to see the JSON from the file being printed. Try changing the **`users.json`** file and see how it is read again on the next request (when refreshing the website):

```
$ node backend/webfiles.js
```

While useful as an exercise, files are not a proper database to be used in production. As such, we are later going to introduce MongoDB as a database. We are going to run the MongoDB server in Docker, so let's first briefly have a look at Docker.

Introducing Docker, a platform for containers

Docker is a platform that allows us to package, manage, and run applications in loosely isolated environments, called **containers**. Containers are lightweight, are isolated from each other, and include all dependencies needed to run an application. As such, we can use containers to easily set up various services and apps without having to deal with managing dependencies or conflicts between them.

NOTE

There are also other tools, such as Podman (which even has a compatibility layer for the Docker CLI commands), and Rancher Desktop, which also supports Docker CLI commands.

We can use Docker locally to set up and run services in an isolated environment. Doing so avoids polluting our host environment and ensures that there is a consistent state to build upon. This consistency is especially important when working in larger development teams, as it ensures that everyone is working with the same state.

Additionally, Docker makes it easy to deploy containers to various cloud services and run them in a **continuous integration/continuous delivery (CI/CD)** workflow.

In this section, we will first get an overview of the Docker platform. Then, we will learn how to create a container and how to access Docker from VS Code. At the end, we will understand how Docker works and how it can be used to manage services.

The Docker platform

The Docker platform essentially consists of three parts:

- **Docker Client**: Can run commands by sending them to the **Docker daemon**, which is either running on the local machine or a remote environment.
- **Docker Host**: Contains the Docker daemon, images, and containers.
- **Docker Registry**: Hosts and stores docker images, extensions, and plugins. By default, the public registry **Docker Hub** will be used to search for images.

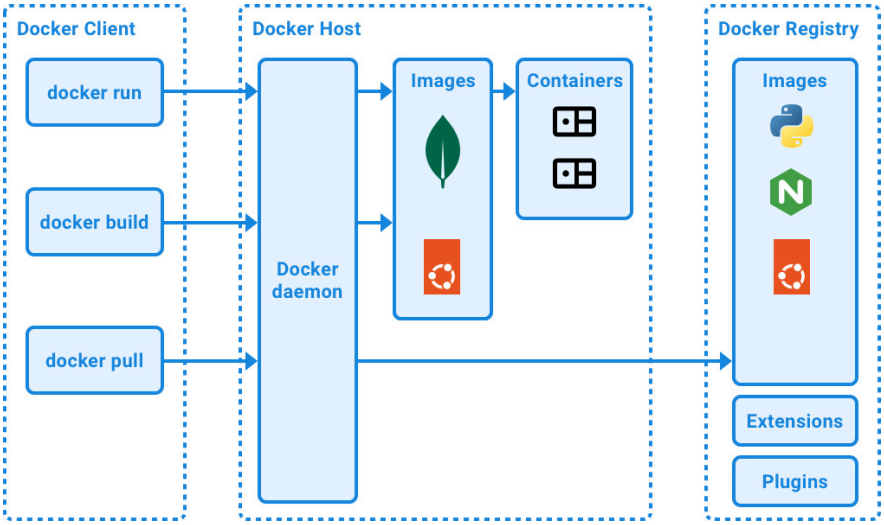

Figure 2.4 – Overview of the Docker platform

Docker images can be thought of as read-only templates and are used to create containers. Images can be based on other images. For example, the `mongo` image, which contains a MongoDB server, is based on the `ubuntu` image.

Docker containers are instances of images. They run an operating system with a configured service (such as a MongoDB server on Ubuntu). Additionally, they can be configured, for example, to forward some ports from within the container to the host, or to mount a storage volume in the container that stores data on the host machine. By default, a container is isolated from the host machine, so if we want to access ports or storage from it on the host, we need to tell Docker to allow this.

Installing Docker

The easiest way to set up the Docker platform for local development is using Docker Desktop. It can be downloaded from the official Docker website (https://www.docker.com/products/docker-desktop/). Follow the instructions to install it and start the Docker engine. After installation, you should have a `docker` command available in your Terminal. Run the following command to verify that it is working properly:

```
$ docker -v
```

This command should output the Docker version, like in the following example:

```
Docker version 24.0.6, build ed223bc
```

After installing and starting Docker, we can move on to creating a container.

Creating a container

Docker Client can instantiate a container from an image via the **`docker run`** command. Let's now create an **`ubuntu`** container and run a shell (**`/bin/bash`**) in it:

```
$ docker run -i -t ubuntu:24.04 /bin/bash
```

NOTE

The **`:24.04`** *string after the image name is called the* **tag**, *and it can be used to pin images to certain versions. In this book, we use tags to pull specific versions of images so that the steps are reproducible even when new versions are released. By default, if no tag is specified, Docker will attempt to use the* **`latest`** *tag.*

A new shell will open. We can verify that this shell is running in the container by executing the following command to see which operating system is running:

```
$ uname -a
```

If you get a version number that ends with **`-linuxkit`**, you have successfully run a command in the container, because LinuxKit is a toolkit to create small Linux VMs!

You can now type the following command to exit the shell and the container:

```
$ exit
```

The following figure shows the result of running these commands:

Figure 2.5 – Running our first Docker container

The **`docker run`** command does the following:

- If you have never run a container based on the **`ubuntu`** image before, Docker will start by pulling the image from the Docker registry (this is equivalent to executing **`docker pull ubuntu`**).
- After the image is downloaded, Docker creates a new container (the equivalent to executing **`docker container create`**).
- Then, Docker configures a read-write filesystem for the container and creates a default network interface.
- Finally, Docker starts the container and executes the specified command. In our case, we specified the **`/bin/bash`** command. Because we passed the **`-i`** (keeps **`STDIN`** open) and **`-t`** (allocates a pseudo-tty) options, Docker attaches the container's shell to our currently running Terminal, allowing us to use the container as if we were directly accessing a Terminal on our host machine.

As we can see, Docker is very useful for creating self-contained environments for our apps and services to run in. Later in this book, we are going to learn how to package our own apps in Docker containers. For now, we are only going to use Docker to run services without having to install them on our host system.

Accessing Docker via VS Code

We can also access Docker via the VS Code extension we installed in ***Chapter 1***, *Preparing for Full-stack Development*. To do so, click the Docker icon in the left sidebar of VS Code. The Docker sidebar will open, showing you a list of containers, images, registries, networks, volumes, contexts, and relevant resources:

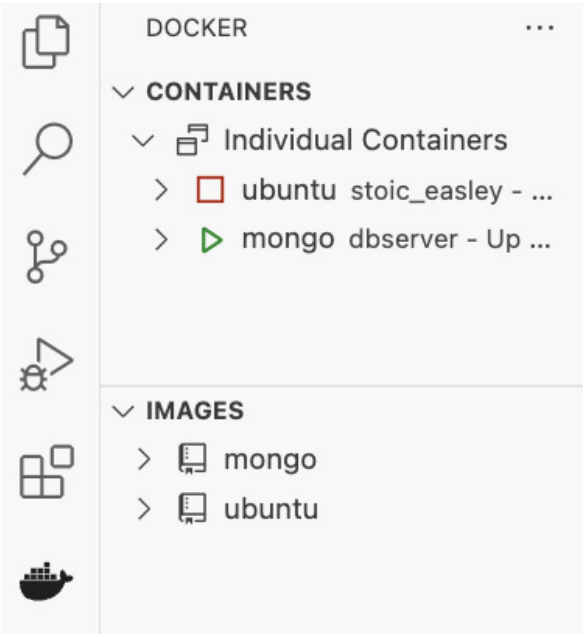

Figure 2.6 – The Docker sidebar in VS Code

Here, you can see which containers are stopped and which ones are running. You can right-click on a container to start, stop, restart, or remove it. You can also view its logs to debug what is going on inside the container. Additionally, you can attach a shell to the container to get access to its operating system.

Now that we know the essentials of Docker, we can create a container for our MongoDB database server.

Introducing MongoDB, a document database

MongoDB, at the time of writing, is the most popular NoSQL database. Unlike **Structured Query Language** (**SQL**) databases (such as MySQL or PostgreSQL), NoSQL means that the database specifically does not use SQL to query the database. Instead, NoSQL databases have various other ways to query the database and often have a vastly different structure of how data is stored and queried.

The following main types of NoSQL databases exist:

- Key-value stores (for example, Valkey/Redis)
- Column-oriented databases (for example, Amazon Redshift)
- Graph-based databases (for example, Neo4j)
- Document-based databases (for example, MongoDB)

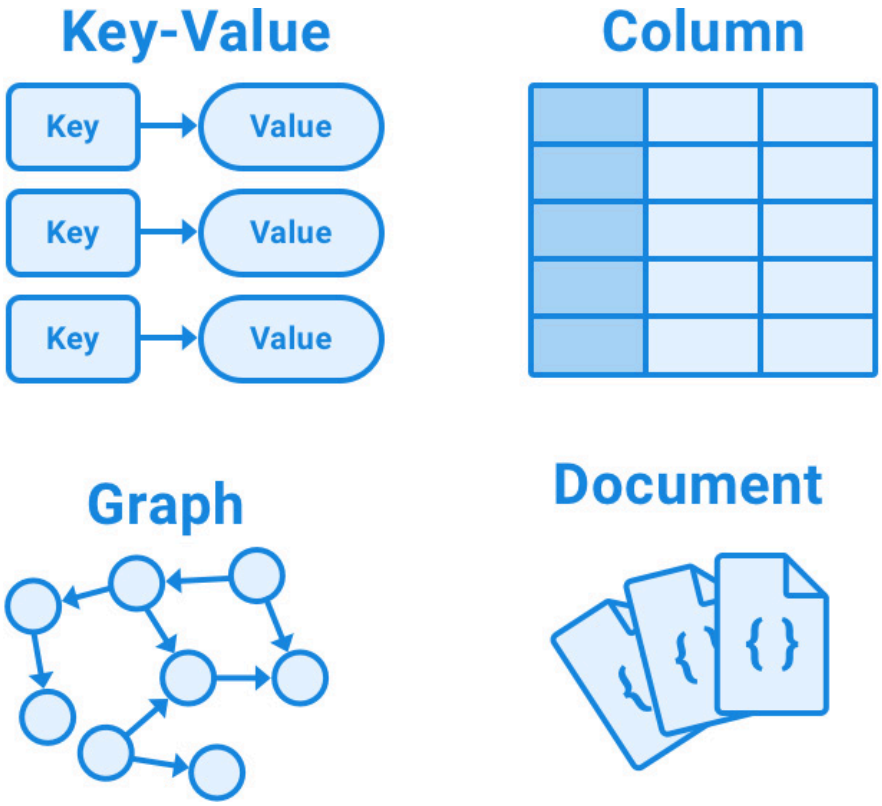

Figure 2.7 – Overview of NoSQL databases

MongoDB is a document-based database, which means that each entry in the database is stored as a document. In MongoDB, these documents are basically JSON objects (internally, they are stored as BSON – a binary JSON format to save space and improve performance, among other advantages). Instead, SQL databases store data as rows in tables. As such, MongoDB provides a lot more flexibility. Fields can be freely added or left out in documents. The downside of such a structure is that we do not have a consistent schema for documents. However, this can be solved by using libraries, such as Mongoose, which we will learn about in ***Chapter 3***, *Implementing a Backend Using Express, Mongoose ODM, and Jest*.

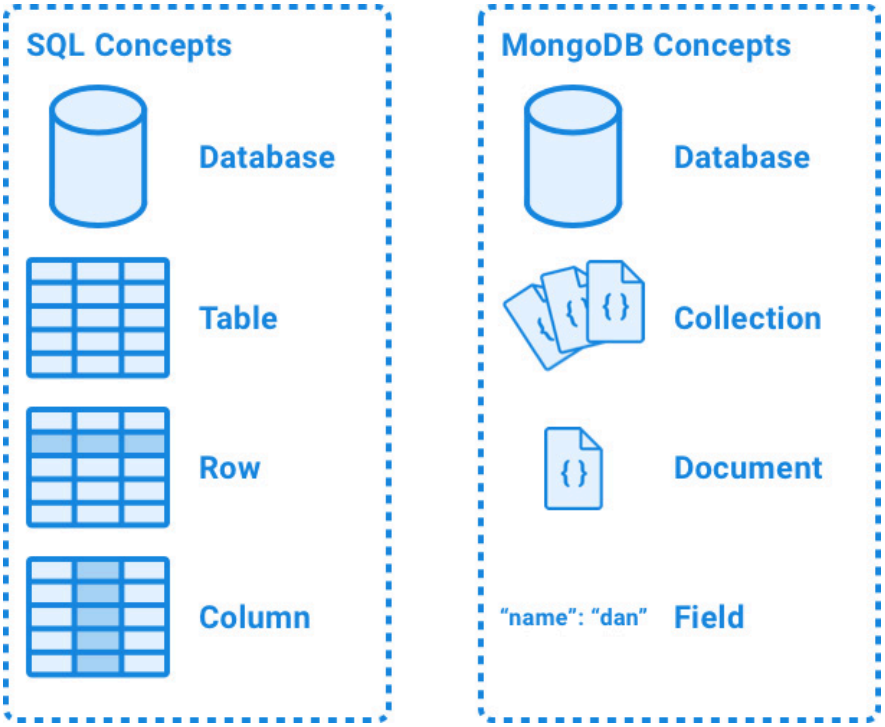

Figure 2.8 – Comparison between MongoDB and SQL databases

MongoDB is also based on a JavaScript engine. Since version 3.2, it has been using SpiderMonkey (the JavaScript engine that Firefox uses) instead of V8. Nevertheless, this still means we can execute JavaScript code in MongoDB. For example, we can use JavaScript in the **MongoDB Shell** to help with administrative tasks. Again, we must be careful with this, though, as the MongoDB environment is vastly different from a browser or Node.js environment.

In this section, we will first learn how to set up a MongoDB server using Docker. Then, we will learn more about MongoDB and how to access it directly using the MongoDB Shell for the administration of our database and the data. We are also going to learn how to use VS Code to access MongoDB. At the end of this section, you will have an understanding of how CRUD operations work in MongoDB.

NOTE

CRUD is an acronym for create, read, update, and delete, which are the common operations that backend services usually provide.

Setting up a MongoDB server

Before we can start using MongoDB, we need to set up a server. Since we already have Docker installed, we can make things easier for ourselves by running MongoDB in a Docker container. Doing so also allows us to have separate, clean MongoDB instances for our apps by creating separate containers. Let's get started with the steps:

1. Make sure Docker Desktop is running and Docker is started. You can verify this by running the following command, which lists all running containers:

```
$ docker ps
```

   If Docker is not started properly, you will get a **Cannot connect to the Docker daemon** error. In that case, make sure Docker Desktop is running and the Docker Engine is not paused.
   If Docker is started properly, you will see the following output:

```
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

   If you already have some containers running, it will be followed by a list of started containers.
2. Run the following Docker command to create a new container with a MongoDB server:

```
$ docker run -d --name dbserver -p 27017:27017 --restart unless-stopped mongo:6.0.4
```

   The **`docker run`** command creates and runs a new container. The arguments are as follows:
   1. **`-d`**: Runs the container in the background (daemon mode).
   2. **`--name`**: Specifies a name for the container. In our case, we named it **`dbserver`**.
   3. **`-p`**: Maps a port from the container to the host. In our case, we map the default MongoDB server port **`27017`** in the container to the same port on our host. This allows us to access the MongoDB server running within our container from outside of it. If you already have a MongoDB server running on that port, feel free to change the first number to some other port, but make sure to also adjust the port number from **`27017`** to your specified port in the following guides.
   4. **`--restart unless-stopped`**: Makes sure to automatically start (and restart) the container unless we manually stop it. This ensures that every time we start Docker, our MongoDB server will already be running.
   5. **`mongo`**: This is the image name. The **`mongo`** image contains a MongoDB server.
3. Install the MongoDB Shell on your host system (not within the container) by following the instructions on the MongoDB website (**https://www.mongodb.com/docs/mongodb-shell/install/**).
4. On your host system, run the following command to connect to the MongoDB server using the MongoDB Shell (**`mongosh`**). After the hostname and port, we specify a database name. We are going to call our database **`ch2`**:

```
$ mongosh mongodb://localhost:27017/ch2
```

You will see some output from the database server, and at the end, we get a shell running on our selected database, as can be seen by the **ch2>** prompt. Here, we can enter commands to be executed on our database. Interestingly, MongoDB, like Node.js, also exposes a JavaScript engine, but with yet another different environment. So, we can run JavaScript code, such as the following:

```
ch2> console.log("test")
```

The following figure shows JavaScript code being executed in the MongoDB Shell:

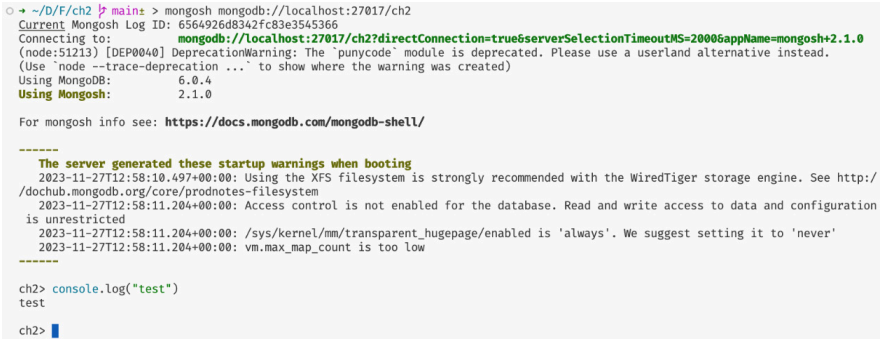

Figure 2.9 – Connecting to our MongoDB database server running in a Docker container

Now that we have a shell connected to our MongoDB database server, we can start practicing running commands directly on the database.

Running commands directly on the database

Before we get started creating a backend service that interfaces with MongoDB, let's spend some time getting familiar with MongoDB itself via the MongoDB Shell. The MongoDB Shell is very important for debugging and doing maintenance tasks on the database, so it is a good idea to get to know it well.

Creating a collection and inserting and listing documents

Collections in MongoDB are the equivalent of tables in relational databases. They store documents, which are like JSON objects. To make it easier to understand, a collection can be seen as a very large JSON array containing JSON objects. Unlike simple arrays, collections support the creation of indices, which speed up the lookup of certain fields in docu-

ments. In MongoDB, a collection is automatically created when we attempt to insert a document into it or create an index for it.

Let's use the MongoDB Shell to insert a document into our database in the **`users`** collection:

1. To insert a new user document into the **`users`** collection, run the following command in the MongoDB Shell:

   ```
   > db.users.insertOne({ username: 'dan', fullName: 'Daniel Bugl', age: 26 })
   ```

   Commands that access the database are prefixed with **`db`**, then the collection name follows, and finally comes the operation, all separated by periods.

NOTE

While **`insertOne()`** *allows us to insert a single document into the collection, there is also an* **`insertMany()`** *method, where we can pass an array of documents to add to the collection.*

2. We can now list all documents from the **`users`** collection by running the following command:

   ```
   > db.users.find()
   ```

   Doing so will return an array with our previously inserted document:

   ```
   [
     {
       _id: ObjectId("6405f062b0d06adeaeefc3bc"),
       username: 'dan',
       fullName: 'Daniel Bugl',
       age: 26
     }
   ]
   ```

As we can see, MongoDB automatically created a unique ID (**`ObjectId`**) for our document. This ID consists of 12 bytes in hexadecimal format (so each byte is displayed as two characters). The bytes are defined as follows:

- The first 4 bytes are a timestamp, representing the creation of the ID measured in seconds since the Unix epoch
- The next 5 bytes are a random value unique to the machine and currently running database process
- The last 3 bytes are a randomly initialized incrementing counter

NOTE

The way **`ObjectId`** *identifiers are generated in MongoDB ensures that IDs are unique, avoiding ID collisions even when two ids are generated at the same time from different instances, without requiring a form of communication between the instances, which would slow down the creation of documents when scaling the database.*

Querying and sorting documents

Now that we have inserted some documents, we can query them by accessing different fields from the object. We can also sort the list of documents returned from MongoDB. Follow these steps:

1. Before we get started querying, let's insert two more documents into our **`users`** collection:

```
> db.users.insertMany([
  { username: 'jane', fullName: 'Jane Doe', age: 32 },
  { username: 'john', fullName: 'John Doe', age: 30 }
])
```

2. Now we can start querying for a certain username by using **`findOne`** and passing an object with the **`username`** field. When using **`findOne`**, MongoDB will return the first matching object:

```
> db.users.findOne({ username: 'jane' })
```

3. We can also query for full names, or any other field in the documents from the collection. When using **`find`**, MongoDB will return an array of all matches:

```
> db.users.find({ fullName: 'Daniel Bugl' })
```

4. An important thing to watch out for is that when querying an **`ObjectId`**, we need to wrap the ID string with an **`ObjectId()`** constructor, as follows:

```
> db.users.findOne({ _id: ObjectId('6405f062b0d06adeaeefc3bc') })
```

   Make sure to change the string passed to the **`ObjectId()`** constructor to a valid **`ObjectId`** returned from the previous commands.

5. MongoDB also provides certain query operators, prefixed by **`$`**. For example, we can find everyone above the age of 30 in our collection by using the **`$gt`** operator, as follows:

```
> db.users.find({ age: { $gt: 30 } })
```

You will notice that **`John Doe`** does not get returned, because his age is exactly 30. If we want to match ages greater than or equal to 30, we need to use the **`$gte`** operator.

6. If we want to sort our results, we can use the **`.sort()`** method after **`.find()`**. For example, we can return all items from the **`users`** collection, sorted by age ascending (**`1`** for ascending, **`-1`** for descending):

```
> db.users.find().sort({ age: 1 })
```

Updating documents

To update a document in MongoDB, we combine the arguments from the query and insert operations into a single operation. We can use the same criteria to filter documents as we did for **`find()`**. To update a single field from the document, we use the **`$set`** operator:

1. We can update the **`age`** field for the user with the username **`dan`** as follows:

```
> db.users.updateOne({ username: 'dan' }, { $set: { age: 27 } })
```

NOTE

Just like **`findOne`**, **`updateOne`** *only updates the first matching document. If we want to update all documents that match, we can use* **`updateMany`**.

MongoDB will return an object with information about how many documents matched (**`matchedCount`**), how many were modified (**`modifiedCount`**), and how many were upserted (**`upsertedCount`**).

2. The **`updateOne`** method accepts a third argument, which is an **`options`** object. One useful option here is the **`upsert`** option, which, if set to **`true`**, will insert a document if it does not exist yet, and update it if it does exist already. Let's first try to update a non-existent user with **`upsert: false`**:

```
> db.users.updateOne({ username: 'new' }, { $set: { fullName: 'New User' } })
```

3. Now we set **`upsert`** to **`true`**, which inserts the user:

```
> db.users.updateOne({ username: 'new' }, { $set: { fullName: 'New User' } }, { upsert
```

NOTE

If you want to remove a field from a document, use the **`$unset`** *operator. If you want to replace the whole document with a new document, you can use the* **`replaceOne`** *method and pass a new document as the second argument to it.*

Deleting documents

To delete documents from the database, MongoDB provides the **`deleteOne`** and **`deleteMany`** methods, which have a similar API to the **`updateOne`** and **`updateMany`** methods. The first argument is, again, used to match documents.

Let's say the user with the username **`new`** wants to delete their account. To do so, we need to remove them from the **`users`** collection. We can do so as follows:

```
> db.users.deleteOne({ username: 'new' })
```

That's all there is to it! As you can see, it is very simple to do CRUD operations in MongoDB if you already know how to work with JSON objects and JavaScript, making it the perfect database for a Node.js backend.

Now that we have learned how to access MongoDB using the MongoDB Shell, let's learn about accessing it from within VS Code.

Accessing the database via VS Code

Up until now, we have been using the Terminal to access the database. If you remember, in ***Chapter 1***, *Preparing for Full-stack Development*, we installed a MongoDB extension for VS Code. We can now use this extension to access our database in a more visual way:

1. Click on the MongoDB icon on the left sidebar (it should be a leaf icon) and click on the **Add Connection** button:

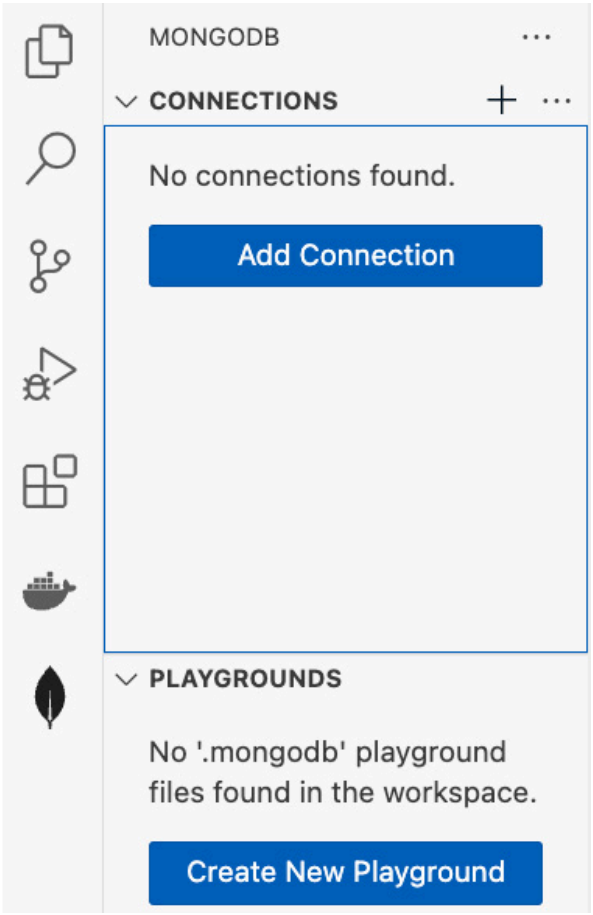

Figure 2.10 – The MongoDB sidebar in VS Code

2. A new **MongoDB** tab will open up. In this tab, click on **Connect** in the **Connect with Connection String** box:

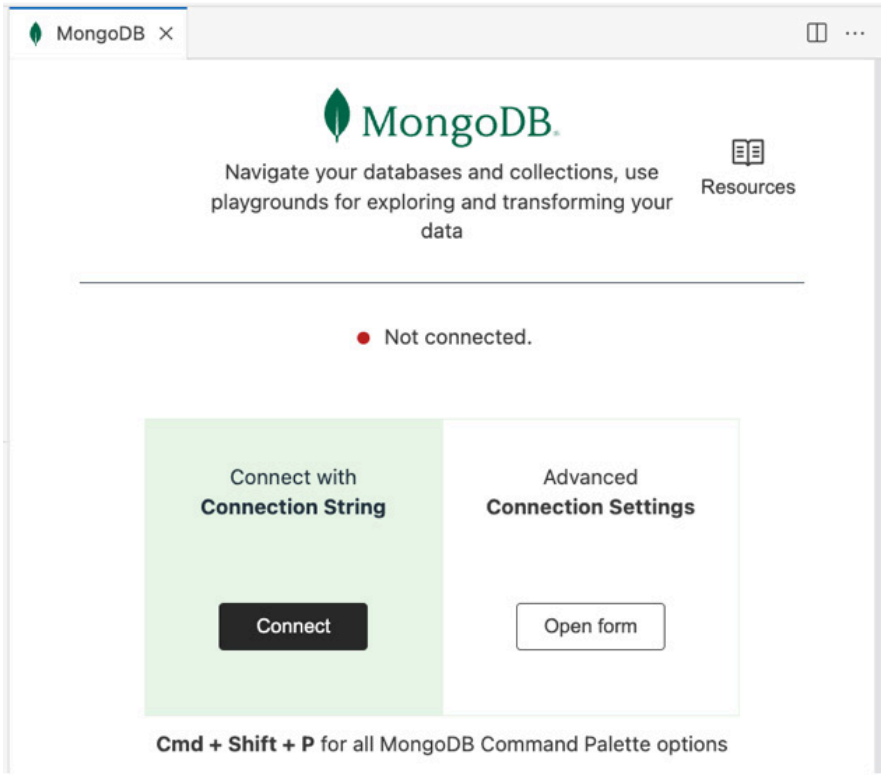

Figure 2.11 – Adding a new MongoDB connection in VS Code

3. A popup should open at the top. In this popup, enter the following connection string to connect to your local database:

```
mongodb://localhost:27017/
```

4. Press *Return/Enter* to confirm. A new connection will be listed in the MongoDB sidebar. You can browse the tree to view databases, collections, and documents. For example, click the first document to view it:

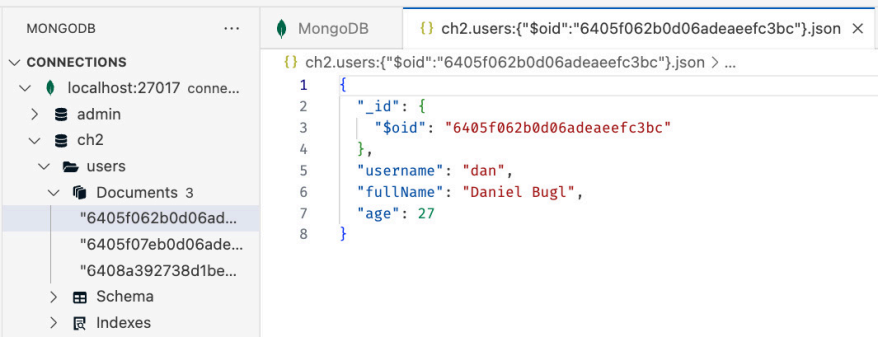

Figure 2.12 – Viewing a document in the MongoDB extension in VS Code

TIP

You can also directly edit a document by editing a field in VS Code and saving the file. The updated document will automatically be saved to the database.

The MongoDB extension is very useful for debugging our database, as it lets us visually spot problems and quickly make edits to documents. Additionally, we can right-click on **Documents** and **Search for documents...** to open a new window where we can run MongoDB queries, just like we did in the Terminal. The queries can be executed on the database by clicking on the **Play** button in the top right. You may need to confirm a dialog with **Yes**, and then the results will show in a new pane, as can be seen in the following screenshot:

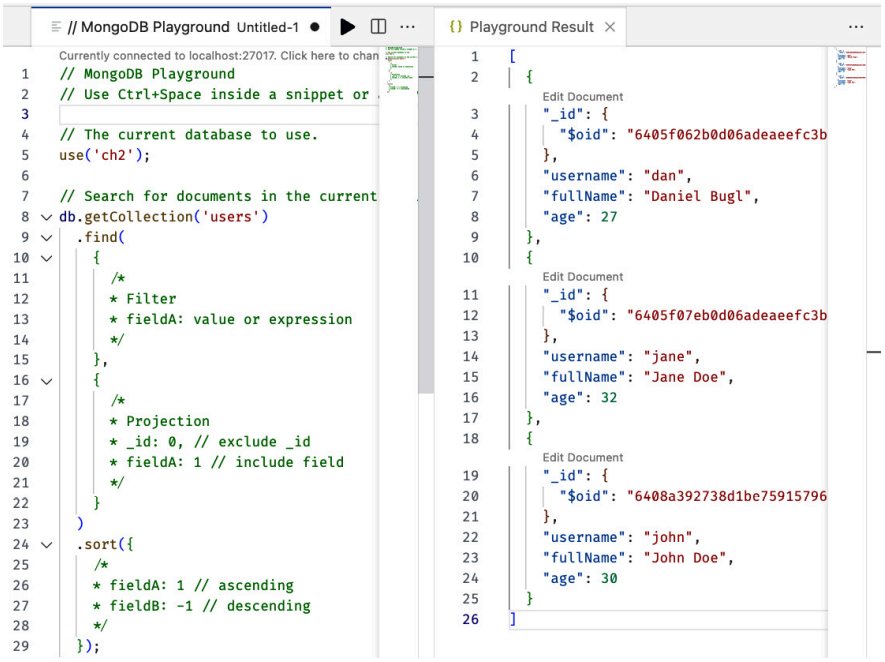

Figure 2.13 – Querying MongoDB in VS Code

Now that we have learned the basics of using and debugging MongoDB databases, we can start integrating our database in a Node.js backend service, instead of simply storing and reading information from files.

Accessing the MongoDB database via Node.js

We are now going to create a new web server that, instead of returning users from a JSON file, returns the list of users from our previously created **`users`** collection:

1. In the **`ch2`** folder, open a Terminal. Install the **`mongodb`** package, which contains the official MongoDB driver for Node.js:

```
$ npm install mongodb@6.3.0
```

2. Create a new **`backend/mongodbweb.js`** file and open it. Import the following:

```
import { createServer } from 'node:http'
import { MongoClient } from 'mongodb'
```

3. Define the connection URL and database name and then create a new MongoDB client:

```
const url = 'mongodb://localhost:27017/'
```

```
const dbName = 'ch2'
const client = new MongoClient(url)
```

4. Connect to the database and log a message after we are connected successfully, or when there is an error with the connection:

```
try {
  await client.connect()
  console.log('Successfully connected to database!')
} catch (err) {
  console.error('Error connecting to database:', err)
}
```

5. Next, create an HTTP server, like we did before:

```
const server = createServer(async (req, res) => {
```

6. Then, select the database from the client, and the **`users`** collection from the database:

```
  const db = client.db(dbName)
  const users = db.collection('users')
```

7. Now, execute the **`find()`** method on the **`users`** collection. In the MongoDB Node.js driver, we also need to call the **`toArray()`** method to resolve the iterator to an array:

```
  const usersList = await users.find().toArray()
```

8. Finally, set the status code and response header, and return the users list:

```
  res.statusCode = 200
  res.setHeader('Content-Type', 'application/json')
  res.end(JSON.stringify(usersList))
})
```

9. Now that we have defined our server, copy over the code from before to listen to **`localhost`** on port **`3000`**:

```
const host = 'localhost'
const port = 3000
server.listen(port, host, () => {
  console.log(`Server listening on http://${host}:${port}`)
})
```

10. Run the server by executing the script:

```
$ node backend/mongodbweb.js
```

11. Open **`http://localhost:3000`** in your browser and you should see the list of users from our database being returned:

Figure 2.14 – Our first Node.js service retrieving data from a MongoDB database!

As we have seen, we can use similar methods that we have used in the MongoDB Shell in Node.js as well. However, the APIs of the **`node:http`** module and the **`mongodb`** package are very low-level, requiring a lot of code to create an HTTP API and talk to the database.

In the next chapter, we are going to learn about libraries that abstract these processes to allow for easier creation of HTTP APIs and handling of the database. These libraries are Express and Mongoose. Express is a web framework that allows us to easily define API routes and handle requests. Mongoose allows us to create schemas for documents in our database to more easily create, read, update, and delete objects.

Summary

In this chapter, we learned how to use Node.js to develop scripts that can run on a server. We also learned how to create containers with Docker, and how MongoDB works and can be interfaced with. At the end of this chapter, we even successfully created our first simple backend service using Node.js and MongoDB!

In the next chapter, ***Chapter 3***, *Implementing a Backend Using Express, Mongoose ODM, and Jest*, we are going to learn how to put together what we learned in this chapter to extend our simple backend service to a production-ready backend for a blog application.